

Aegis Markets: A High-Throughput Prediction Market Platform

Project Goal: To develop a highly scalable and resilient **centralized prediction market platform**, focusing on the core technical challenge of handling thousands of simultaneous trade requests and ensuring rapid, accurate settlement, while exclusively supporting cryptocurrency deposits.

1. EXECUTIVE SUMMARY

1.1 Problem Statement & Solution Overview

Current decentralized prediction market platforms often struggle with technical limitations that restrict mainstream adoption. Key pain points include **transaction latency** and high processing costs inherent to Layer 1 (L1) blockchain solutions. While Layer 2 (L2) solutions improve speed, the underlying complexity of on-chain settlement can still present challenges. **Aegis Markets** addresses these issues by adopting a **centralized architecture** that leverages the user experience of Web3 (crypto-native deposits) with the raw performance of a traditional high-frequency trading system.

When market-moving events occur, the system must handle massive, instantaneous spikes in transaction volume (thousands of trades per minute) while maintaining perfect **data consistency** and preventing race conditions in user balances and liquidity pools. Aegis Markets proposes a solution using a **microservices architecture** designed for horizontal scaling, prioritizing speed and transactional integrity. The core innovation lies in the **Transaction Processor Service**, written in Go (Golang) and buffered by a **Kafka** message queue. This decouples transaction ingestion from processing and ensures high-speed, ordered writes to a robust relational database (**PostgreSQL**). This approach delivers the necessary scalability and payout accuracy essential for a premier prediction market platform, offering a seamless user experience without the gas fees or network congestion of purely on-chain alternatives.

1.2 Resource Requirements

Resource Type	Component	Justification
Compute	Microservices (Transaction Processor, etc.)	Cloud-native, containerized environment (AWS ECS/EKS) for horizontal scaling and load balancing.

Data Storage	PostgreSQL (AWS RDS)	Industry-standard relational database providing strong transactional ACID guarantees and high availability.
Data Buffer	Apache Kafka	Acts as a high-throughput, ordered queue to buffer incoming trade requests, preventing the database from becoming a bottleneck during peak load.
Caching	Redis	Used for storing real-time market odds and user balances to minimize read latency on the primary database.
Personnel	Developers / Engineers	Required skillsets: Go/Node.js, React/TypeScript, DevOps (Docker/AWS), and Database Management (PostgreSQL).

2. SYSTEM ARCHITECTURE

2.1 Architecture Overview

The platform is designed around a **Microservices Architecture** pattern, chosen specifically for its ability to isolate high-load services (like the Transaction Processor) and scale them independently.

- **Client (React/TypeScript):** The single-page application (SPA) handling the user interface. It connects via REST for read data and uses **Server-Sent Events (SSE)** for real-time odds updates. It integrates with crypto wallets like **MetaMask** for authentication and to initiate deposits. - schatgpt v0
- **Market Service:** A lightweight service (Node.js/Express) managing the static state of markets (e.g., creation, resolution criteria, titles). Market creation will initially be curated to ensure quality and unambiguous resolution sources. **Garri**

- **Transaction Processor (CORE Service):** A high-performance service **written in Go** that handles the core business logic of trading. It validates user funds, calculates the price using the platform's Automated Market Maker (AMM) logic, and publishes the transaction request to the Kafka queue. - **YH (downie version)**
- **Kafka Message Queue:** A durable, ordered queue acting as the buffer between the high-volume Transaction Processor and the persistent data layer (PostgreSQL), ensuring transactions are processed sequentially and reliably. (**JSON**)
- **Settlement Service (CRITICAL Service):** Reads completed market results from the Resolution Service and processes the final payout logic, calculating profits/losses and initiating user balance updates in the database. - **EC (make it back in the hood version)**
- **Resolution Service:** A **centralized service** that integrates with trusted, real-world data sources (e.g., sports score APIs) to verify the final outcome of a market and feed the result to the Settlement Service. This serves as the centralized equivalent of a decentralized oracle. - **jered**

<https://excalidraw.com/#room=d0d0c80755be108b09aa.JgLi0RjEWdVj9ACTT5Q45Q>

2.2 Technology Stack

Frontend

- **Framework Selection: React with TypeScript.** Justification: Provides a robust and scalable component-based architecture for complex, data-intensive UIs.
- **State Management: Redux Toolkit.** Chosen for its efficiency and suitability for managing complex, real-time data flows (market odds, user balances).
- **UI/UX Approach:** Responsive design using **Tailwind CSS** to ensure a performant and intuitive experience on both desktop and mobile devices.

Backend

- **Language/Framework: Go (Golang)** for the Transaction Processor (critical for concurrency and low latency) and **Node.js/Express** for less performance-intensive services (Market Service).
- **API Design: RESTful APIs** for standard data retrieval and manipulation. **Server-Sent Events** for real-time pushing of price updates, odds changes, and trade confirmations to the client.
- **Service Architecture:** Microservices deployed as Docker containers.

Database

- **Type and Justification: PostgreSQL** (Relational SQL Database). Justification: PostgreSQL is the most robust and popular open-source SQL database, offering industry-leading **strong ACID guarantees** and reliability, which are non-negotiable for financial and transactional data integrity. Its advanced indexing and transactional features are critical for maintaining state consistency in the LiquidityPool and Transaction

tables.

- **Scaling Strategy:** Will utilize **AWS RDS Multi-AZ** deployment for high availability and **Read Replicas** for handling high read traffic.
- **Backup Approach:** Automated daily snapshots and point-in-time recovery capabilities managed by **AWS RDS (Relational Database Service)**.

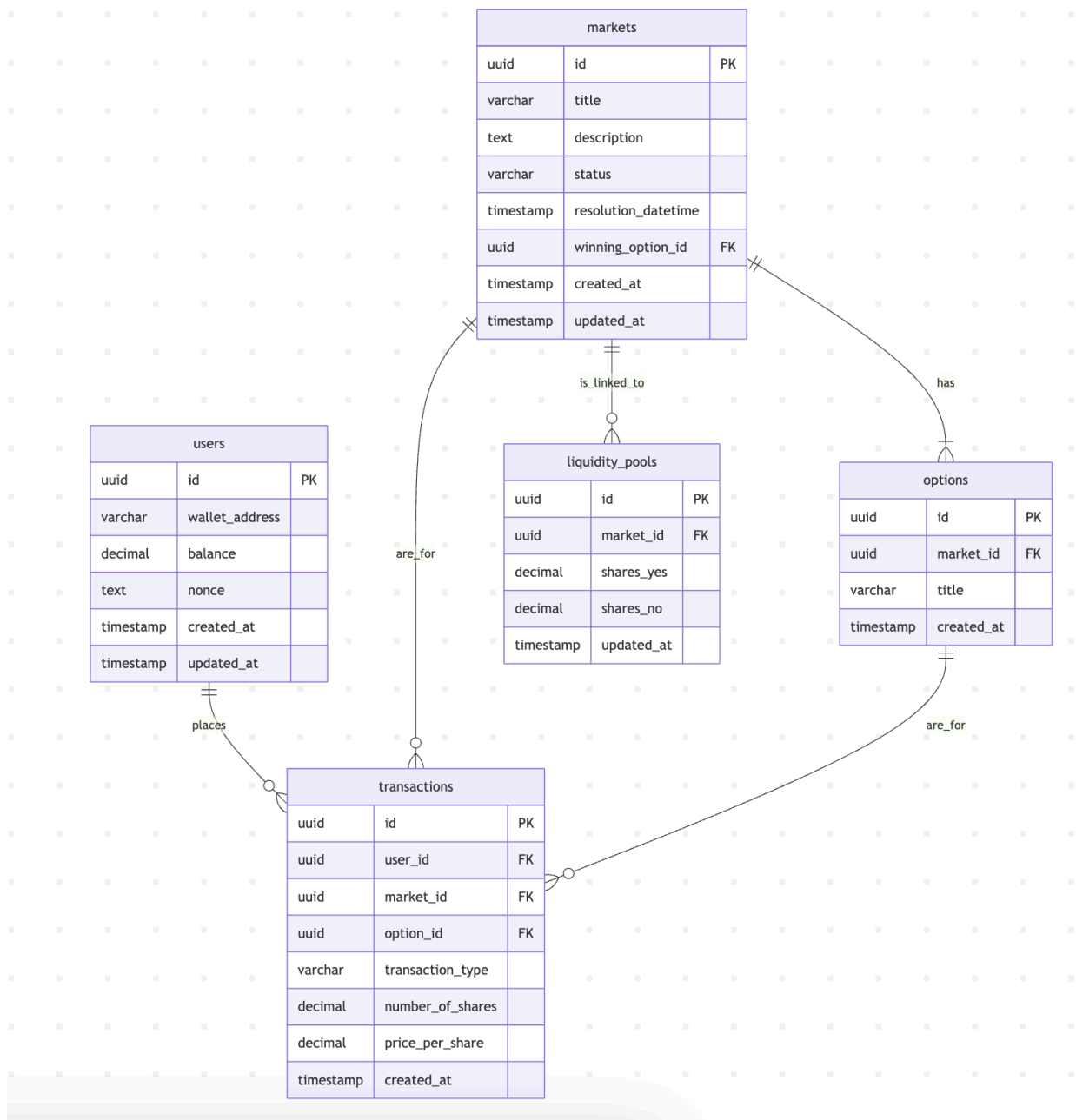
Infrastructure

- **Cloud Provider:** **AWS (Amazon Web Services)**.
- **Deployment Strategy:** Services are deployed as **Docker containers** managed by **AWS ECS (Elastic Container Service)**.
- **CI/CD Approach:** **GitHub Actions** for automated build, testing, containerization, and deployment upon successful merges to the main branch.

3. DATA & SECURITY

3.1 Data Architecture

Simplified Entity Relationship Diagram (ERD)



(Simplified ERD showing core entities and one-to-many relationships)

Core Entities and Relationships

- **Market/Option:** A Market has multiple Options (e.g., YES and NO). The Market is the core entity that dictates the event and resolution.
- **User/Transaction:** Users place many Transactions. The Transaction table is the **highest-write table** and is indexed heavily by marketId and userId.
- **Market/LiquidityPool:** Each Market is linked to exactly one LiquidityPool, which stores the current state and parameters for the Automated Market Maker (AMM) function. This

centralized table is the equivalent of an on-chain smart contract liquidity pool and ensures that concurrent trade calculations are atomic.

Data Flow Patterns

Trade requests are handled via an **Asynchronous/Queued Pattern**. A trade is published to Kafka by the Transaction Processor and consumed by a dedicated Kafka consumer pool which performs the actual database write and balance update. This prevents client-side timeout errors during heavy load and ensures transactions are processed in the order they were received.

Caching Strategy

Redis is implemented as a fast-read cache layer.

- **Write-Through/Write-Back:** User balances and the critical LiquidityPool state are primarily updated in PostgreSQL, and then asynchronously pushed to Redis for fast reads.
- **Time-to-Live (TTL):** Short TTLs (e.g., 30 seconds) are used for less critical data like the list of open markets.

3.2 Security Framework

- FUCK U

API Security

- **Input Validation:** Strict sanitization and validation of all incoming JSON payloads to prevent common vulnerabilities like SQL Injection and Cross-Site Scripting (XSS).

4. IMPLEMENTATION

4.1 User Journey (Core Features)

1. **Placing a Trade (High-Concurrency Path):**
 - User clicks "Buy" on an Option in the UI.
 - Client sends request (OptionID, Amount)
 - Authentication with JWT and routes to the Transaction Processor (Go service).
 - Processor verifies available funds in the user's account (from the database) and calculates the new price using the AMM formula.
 - Processor publishes the complete transaction payload to the **Kafka Queue**.
 - Processor immediately sends a preliminary confirmation back to the client via **Server-Sent Events** (Latency Target: ms).
 - The Kafka Consumer/DB Worker reads the queue and commits the atomic write to PostgreSQL (updating User Balance and LiquidityPool state).
 - Hiding markets until 100k liquidity ensures early trades **don't get unfair pricing**.
 - **Payout Visibility Gate:** Payout projections and potential returns will only be displayed once the liquidity pool exceeds the **100k threshold**, ensuring price signals

and odds are meaningful and not distorted by low-liquidity volatility.

2. Market Creation and Listing:

- A platform administrator submits a new market proposal via an internal UI.
- Market Service validates all fields and writes the new Market and Option records to PostgreSQL, initializing the LiquidityPool with starting parameters.

3. Real-Time Odds Monitoring:

- User navigates to a Market Detail page.
- Client opens a persistent connection for **Server-Sent Events** to the Market Service.
- The Market Service monitors the **Redis** cache for price changes.
- Upon a price change (triggered by a committed trade), the Market Service pushes the new odds/price data to all connected clients.

4. Market Settlement and Payout (Finality Path):

- Resolution Date passes. The Resolution Service fetches the final outcome (e.g., "Yes" or "No") from a trusted external API.
- The Settlement Service locks the Market status to 'Resolving'.
- It queries all outstanding Transaction records for that market.
- It executes the profit/loss calculation, updates the User balances in the database, and finally sets the Market status to 'Resolved'. A correct share is valued at **\$1.00**, while an incorrect share is valued at **\$0**.

4.2 Technical Specifications

Feature	Description	Technical Dependency	Performance Requirements
AMM-Based Trading	Implement a Constant Sum Market Maker (CSMM) , which is ideal for binary outcome prediction markets. The formula ensures the sum of outcome prices remains constant.	Go (for high-speed calculation), PostgreSQL (for atomic state).	Trade calculation ms.
Real-time Price Feed	Push price and odds updates instantly to the UI without polling.	Server-Sent Events, Redis (caching).	Update latency ms.
High-Volume	System must ingest	Go/Kafka/PostgreS	Sustain 1,000

Transaction	and process massive, simultaneous trade requests.	QL.	trades/minute.
Secure Settlement	Guaranteed and verifiable payout based on a verified external data source.	Settlement Service logic, PostgreSQL ACID properties.	Zero state inconsistency (payout errors).

Timeline: (12 Days Capstone)

- **Days 1-3:** Infrastructure Setup (AWS/ECS/Docker), Core Data Model (PostgreSQL), Authentication MVP (**SIWE**), Market Service (CRUD).
- **Days 4-6:** Frontend MVP (Trade Placement UI), Initial Transaction Processor (Node.js/Sequential), Kafka integration.
- **Days 7-9: Refactoring for Scale:** Re-implement Transaction Processor in **Go**, PostgreSQL scaling/tuning, **Server-Sent Events** implementation, Load Testing (P95 latency optimization).
- **Days 10-12:** Settlement Service, Resolution Service integration, Comprehensive Testing (Unit/Integration/Load), Documentation, and Final Polish.

4.3 Technical Risks

Risk Category	Technical Risk	Mitigation Strategy
Performance Bottlenecks	Transaction Processor cannot maintain high throughput (tx/min) during peak load.	Utilize Go's concurrency model for the core processor. Use Kafka as a pressure relief buffer. Scale PostgreSQL by implementing Read Replicas .
Scaling Challenges	High write latency on the Transaction or LiquidityPool tables.	Implement aggressive indexing on critical foreign keys. Perform constant load testing (e.g., with k6) and database query optimization.

Data Consistency (Critical)	Race conditions leading to incorrect user balances or pool state during concurrent writes.	Enforce Strong ACID transactional guarantees using PostgreSQL's SERIALIZABLE isolation level for all balance/pool updates. Use the Kafka queue to enforce strict transaction ordering.
------------------------------------	--	---

5. SUCCESS METRICS

5.1 Performance Metrics

These metrics define the technical success of the high-throughput architecture:

- **Transaction Throughput:** The system must sustain and successfully process **1,000 trades per minute** during a simulated load test (10-minute duration).
- **End-to-End Latency:** The 99th percentile (P99) end-to-end latency for a trade request (from client submission to receipt of the initial success confirmation) must remain below **300 milliseconds**.
- **Database Write Latency:** The 95th percentile (P95) write latency on the Transaction table must remain below **50ms** during peak load simulations.
- **Real-Time Data Latency: Server-Sent Event** updates for market odds must be delivered to clients within **100ms** of the corresponding trade transaction being committed to the database.

5.2 Quality Metrics

These targets define the quality assurance and reliability goals:

- **Test Coverage:** Achieve a minimum of **85% unit and integration test coverage** for the critical business logic in the Transaction Processor (Go) and Settlement Service to prevent logic bugs in the AMM and payout calculations.
- **Zero Payout Errors:** The Settlement Service must resolve test markets with the correct, mathematically verified final payout amounts, ensuring perfect transactional consistency.
- **Service Uptime:** Achieve a target of **99.9%** availability for transaction Processor, and Database cluster during the testing period.
- **Code Quality:** Maintain a high standard of code quality verified by static analysis tools (e.g., SonarQube or linters), aiming for zero critical/major technical debt issues in the core services.